

TP1 Report: MLP with PyTorch on Fashion-MNIST

Tayeb Madani and Mohamed Ilyes Arabet

1. Part 1: Baseline model

Our model is an MLP with an input of 784 values (the 28x28 image flattened), two hidden layers of 256 and 128 neurons with ReLU, and an output layer of 10 neurons, one for each class. The output layer gives raw scores (logits) with no softmax. The model has 235,146 parameters in total.

We used CrossEntropyLoss because this is a classification problem with 10 classes. It applies softmax to the logits and then takes minus the log of the probability given to the true class, so the model is punished a lot when it is confident and wrong. MSE is made for regression, so it is not the right choice here.

We trained with Adam, learning rate 1e-3, no weight decay, batch size 64, for 15 epochs, without dropout or batch norm. The data was split into 48,000 training, 12,000 validation and 10,000 test images.

The training loss goes down during all 15 epochs (from 0.479 to 0.158) and training accuracy goes from 82.6% to 94.0%. Validation accuracy rises quickly to 88.7% at epoch 5 and then stays between 88.4% and 89.2%. The validation loss is lowest at epoch 5 (0.319) and then goes back up to 0.368. At the last epoch the gap between training and validation accuracy is 5.1 points, and it has been growing during training. So after about epoch 5 the model starts to overfit: it keeps getting better on the training images but not on new ones. The test accuracy is 88.39%.

2. Experiment 1: Model architecture

Config	Train acc (ep15)	Val acc (ep15)	Gap	Test acc	Best val
Micro [4]	82.80%	81.80%	1.00	80.78%	81.80%
Small [64]	92.22%	88.44%	3.78	87.75%	88.69%
Baseline [256, 128]	94.10%	88.85%	5.25	88.02%	89.26%
Large [512, 256]	94.32%	89.15%	5.17	88.78%	89.20%

Underfitting. Micro [4] almost stops improving after epoch 5 and stays around 82% training accuracy, and its gap is only 1 point. A small gap does not mean the model generalises well here. Training and validation accuracy are both low because 4 neurons are not enough to learn the task. The gap is small only because the model cannot even fit the training data.

Overfitting. Small, Baseline and Large all have a gap that grows over the 15 epochs. In the figure “Exp. 1 — Train-Validation Accuracy Gap: Architecture” all curves start below zero at epoch 1 and then mostly go up from one epoch to the next, reaching 3.78 for Small, 5.25 for Baseline and 5.17 for Large. Training accuracy keeps rising while validation accuracy is flat after about epoch 6 to 8.

Plateau and gap. Validation accuracy already reaches about 88.5% with Small [64] (best 88.69%). Baseline and Large only add around 0.5 point even though they have about 4.6 and 10.5 times more parameters. The extra capacity mostly raises training accuracy. The final gaps are 1.00, 3.78, 5.25 and 5.17, so the gap grows with size up to Baseline and then stays the same. It is not strictly monotonic.

Choice. We keep [256, 128]. It has about the same validation accuracy as Large with less than half the parameters, and it is better than Micro and Small. Its gap of 5.25 is the problem we try to fix in Experiment 3.

3. Experiment 2A: Learning rate with SGD

Learning rate	Final val acc	Best val acc	Test acc
1e-5	24.78%	24.78% (ep 15)	24.81%
1e-3	81.80%	81.80% (ep 15)	80.31%
0.1	88.15%	89.22% (ep 11)	87.74%

With lr 1e-5 the model learns very little. Validation accuracy starts at 6.9% and only reaches 24.8% after 15 epochs, and the loss curve in “2.A — Train Loss: Learning Rate (SGD)” is almost flat. With SGD the update is the learning rate times the gradient, so with such a small value the weights barely move. With lr 1e-3 the model learns but slowly: 55.7% after the first epoch, 80% only at epoch 12 and 81.8% at the end, still going up and with no gap, so it has not converged yet. With lr 0.1 it is much faster: 84.5% after one epoch, 88% around epoch 4 and a best of 89.22% at epoch 11.

For the same 15 epochs, lr 1e-3 ends 6.4 points below lr 0.1. So a bad learning rate with SGD costs a lot: either many more epochs or a worse model. In practice this means we have to try several learning rates, which takes many full trainings.

4. Experiment 2B: SGD compared to Adam

Config	LR	Best val acc	Epoch of best	Epochs to 80%	Epochs to 85%	Epochs to 88%
SGD best	0.1	89.50%	14	1	2	5
Adam best	1e-3	89.28%	13	1	1	4
SGD naive	1e-3	82.17%	15	11	never	never
Adam	0.1	10.20%	13	never	never	never
Adam	1e-5	84.92%	15	4	never	never

SGD gets its best validation accuracy with lr 0.1 (89.50%) and Adam with lr 1e-3 (89.28%). At these best values Adam is a bit faster at the start: 85.6% after the first epoch against 84.3% for SGD, and it reaches 88% one epoch earlier. After that both end at about the same level. So once SGD is tuned, Adam is only slightly faster on this simple MLP. This is because Adam divides the step of each weight by the size of its recent gradients, so every weight gets a good step size from the start without tuning, and it also keeps a moving average of the gradients like momentum.

At the same lr 1e-3 the two optimisers behave very differently. SGD only reaches 82.2% and needs 11 epochs to pass 80%, while Adam has 85.6% after one epoch and ends at 88.9%. The same number does not give the same step: for SGD the step depends on the size of the gradient, while for Adam it is roughly the learning rate for every weight. So a learning rate cannot be reused from one optimiser to another. Adam does not work with every learning rate either: with lr 0.1 it stays at 10%, which is random guessing, because the steps are far too big. But with lr 1e-5 it still gets 84.9% while SGD only got 24.8% with the same learning rate in 2A.

5. Experiment 3: Regularisation

Configuration	Train (ep20)	Val (ep20)	Gap	Test acc	Epoch of best val
No regularisation	95.17%	89.20%	5.97	88.12%	12 (89.38%)
Dropout 0.3	90.67%	89.08%	1.59	88.64%	16 (89.43%)
Weight decay 1e-3	90.68%	88.52%	2.16	87.65%	18 (88.72%)
Dropout + BN	91.63%	89.19%	2.44	89.03%	13 (89.77%)

Dropout randomly turns off about 30% of the hidden neurons at each training step, so the network cannot depend on a few neurons and has to learn more robust features. It is turned off during evaluation. Weight decay adds a penalty on the squared weights to the loss, which keeps the weights small and gives a smoother function that memorises less noise.

Dropout 0.3 gives the smallest gap, 1.59 points compared to 5.97 without regularisation (figure “Exp. 3 — Train–Validation Accuracy Gap: Regularisation”). Dropout with batch norm gives the highest validation accuracy during training (89.77%) and the best test accuracy (89.03%) with a small gap of 2.44. The final validation accuracy is actually close for no regularisation, dropout and dropout with batch norm (89.1% to 89.2%). So the main effect of regularisation here is to stop training accuracy from going too high (95.2% down to about 91%), while the gain on validation is small. Weight decay 1e-3 also reduces the gap, but its validation curve is less stable (it drops to 85.6% at epoch 6) and it ends with the lowest scores, so 1e-3 is probably too strong.

Without regularisation the best validation accuracy comes at epoch 12, and after that only training accuracy improves. With dropout (epoch 16) and weight decay (epoch 18) the best epoch comes later. Training is harder with regularisation, so the model learns more slowly, but validation keeps improving for more epochs. Dropout with batch norm peaks at epoch 13, earlier than dropout alone, because batch norm makes training faster and more stable. So regularisation slows down convergence but makes the extra epochs useful. Without it we would need to stop training early, around epoch 12.